

Lab 1: The Zero-Config Workspace Setup

Step 1.1: Initialize the Monorepo

Run the creation script. When prompted, select Integrated Monorepo (or None for a blank layout) and choose Vite/React or Node to match this setup.

```
# Original: npx create-nx-workspace@latest nx-demo-workspace --preset=apps
pnpm create nx-workspace@latest nx-demo-workspace --preset=apps
```

- Prompt choices: Select integrated style, no cloud (for now), and use your preferred package manager (e.g., npm or pnpm).

Step 1.2: Navigate into the Workspace

```
cd nx-demo-workspace
```

Step 1.3: Generate the React App

Install the React plugin and scaffold the frontend.

```
# Original: npm add -D @nx/react
pnpm add -D -w @nx/react

# Original: npx nx g @nx/react:app apps/frontend --bundler=vite --
e2eTestRunner=none
pnpm nx g @nx/react:app apps/frontend --bundler=vite --e2eTestRunner=none
```

Step 1.4: Generate the Express NodeJS API

Install the Express plugin and scaffold the backend.

```
# Original: npm add -D @nx/express
pnpm add -D -w @nx/express

# Original: npx nx g @nx/express:app apps/backend --e2eTestRunner=none
pnpm nx g @nx/express:app apps/backend --e2eTestRunner=none
```

Step 1.5: Discover Inferred Projects

Nx automatically identifies targets from vite.config.ts and package.json files without requiring individual project.json files. Verify the setup:

```
pnpm nx show projects
```

Lab 2: Interactive Dependency Graph Mapping

Step 2.1: Establish an Application-to-Application Dependency

Simulate a misconfiguration where the backend code mistakenly tries to import configuration details directly from the frontend React environment file.

Open `apps/backend/src/main.ts` in your code editor and append this line at the top:

```
import { defineConfig } from '../../../frontend/vite.config';
```

Step 2.2: Launch the Local Visualizer

Spin up the interactive development server to inspect the workspace layout.

```
pnpm nx graph
```

Your browser will automatically launch a window displaying the interactive map.

Step 2.3: Trace Specific Connection Links

Filter down to target nodes to see how Nx reads file imports:

```
pnpm nx graph --focus=backend
```

Observe the visual line running directly from backend to frontend. This visually highlights why code separation boundaries will be needed later.

Lab 3: Master the Cache & Pipeline Orchestration

Step 3.1: Execute a Workspace-Wide Task

Run a full build operation across all projects simultaneously.

```
pnpm nx run-many -t build
```

Take note of the total execution time printed in your terminal (e.g., 3.45 seconds).

Step 3.2: Trigger Local Computation Caching

Re-run the exact same build command immediately without altering any workspace files:

```
pnpm nx run-many -t build
```

Observe the terminal terminal metrics. The output will show [local cache] and complete in a fraction of a second (e.g., 120ms), proving no code was recompiled unnecessarily.

Step 3.3: Isolate Cache Scope with Non-Code Files

Create or alter a documentation file that has no impact on code output:

```
echo "# Workspace Updates" > README.md
```

Execute the build sweep once more:

```
pnpm nx run-many -t build
```

The tasks will still hit the [local cache] completely because Nx knows changes to README.md do not alter code build hashes.

Step 3.4: Configure the Task Pipeline Dependency

Open the root nx.json file in your editor. Locate or create the targetDefaults configuration block to ensure that running a build on any app automatically builds its dependent projects first:

```
{
  "targetDefaults": {
    "build": {
      "dependsOn": ["^build"],
      "cache": true
    }
  }
}
```

Step 3.5: Verify the Build Sequencing Hierarchy

To confirm that Nx properly understands your configuration sequence without executing the compiler, run a pipeline dry-run:

```
pnpm nx show project backend --graph
```

Lab 4: Extracting and Sharing Monorepos Libraries

Step 4.1: Generate a Shared Utility Library

We will create a clean, reusable TypeScript library designed to host shared types, models, or validation logic.

```
pnpm nx g @nx/js:lib libs/shared-models --bundler=none --
unitTestRunner=jest
```

Step 4.2: Add Shared Logic

Open `libs/shared-models/src/lib/shared-models.ts` in your text editor. Replace its contents with a shared data interface:

```
export interface UserProfile {
  id: string;
  name: string;
  role: 'admin' | 'user';
}
```

Step 4.3: Consume the Shared Library in the Frontend

Open `apps/frontend/src/app/app.tsx` (or your main frontend entry file). Import and use the shared interface. Thanks to Nx, you can use the clean workspace path alias defined automatically in your root `tsconfig.base.json`:

```
import { UserProfile } from '@nx-demo-workspace/shared-models';
const user: UserProfile = { id: '1', name: 'Alex', role: 'admin' };
```

Step 4.4: Verify the New Project Graph

Before checking the graph, clean up the intentional bad import from Lab 2 (remove the `import { defineConfig }...` line from `apps/backend/src/main.ts`). Now, view the clean, modular layout:

```
pnpm nx graph
```

You will now see a visual arrow pointing from frontend down to shared-models.

Lab 5: Differential CI with Affected Commands## Step 5.1: Initialize Git Tracker

Nx calculates changes relative to your Git history. Let's initialize a local repository and commit our baseline workspace state.

```
git init
git config user.name "Trainer"
git config user.email "trainer@example.com"
git add .
git commit -m "feat: complete base setup and shared library"
```

Step 5.2: Create a Feature Isolation Branch

```
git checkout -b feature/update-frontend
```

Step 5.3: Make a Change Only to the Frontend App

Simulate a developer working exclusively on user interface updates. Open `apps/frontend/src/app/app.tsx` and append a basic comment at the bottom:

```
// Minor UI update for Lab 5
```

Step 5.4: Run Affected Command Validation

Ask Nx to calculate exactly what has changed relative to your main branch. It will dynamically deduce that the backend and shared libraries do not need to be validated.

```
pnpm nx affected -t test --base=main
```

Observe your terminal output: Only the frontend test suites will run. The backend and shared libraries are safely skipped, dramatically saving pipeline processing time.

Step 5.5: Visualise Affected Code Subsystems

See a color-coded representation of exactly what your change impacts across the monorepo architecture:

```
pnpm nx graph --affected --base=main
```

In the browser interface, the modified frontend project node will glow red, highlighting its isolation from the rest of the workspace.

Lab 6: Strict Architecture Enforcement (Module Boundaries)

Step 6.1: Install the Nx ESLint Plugin

To enforce architectural boundaries, we need the Nx linting tools. Run the following command to add the linter plugin to your workspace:

```
npm add -D @nx/eslint @nx/eslint-plugin eslint
```

Step 6.2: Assign Tags to Projects

Nx uses tags inside project configuration profiles to identify a project's type or scope. Open your root `nx.json` file and add a tags array to your projects inside the target configurations. If your workspace uses inferred configuration, you can explicitly add or append tags to specific projects by editing your `nx.json` like this:

```
{
  "projects": {
    "frontend": {
      "tags": ["scope:client"]
    },
    "backend": {
      "tags": ["scope:server"]
    },
    "shared-models": {
      "tags": ["scope:shared"]
    }
  }
}
```

Step 6.3: Configure the Enforcement Rules

Open your root `.eslintrc.json` (or create one if it does not exist) and configure the `@nx/enforce-module-boundaries` rule. We will explicitly forbid `scope:server` projects from importing code tagged with `scope:client`.

```

{
  "plugins":["@nx"],
  "rules": {
    "@nx/enforce-module-boundaries": [
      "error",
      {
        "enforceBuildableLibDependency": true,
        "allow": [],
        "depConstraints": [
          {
            "sourceTag": "scope:server",
            "onlyDependOnLibsWithTags": ["scope:shared", "scope:server"]
          },
          {
            "sourceTag": "scope:client",
            "onlyDependOnLibsWithTags": ["scope:shared", "scope:client"]
          }
        ]
      }
    ]
  }
}

```

Step 6.4: Intentionally Break the Boundary Rule

Let's simulate a developer making a mistake by importing a frontend configuration directly into the backend server.

Open `tsconfig.base.json` in the root folder. Add the frontend path to the `compilerOptions.paths` object

```

{
  "compilerOptions": {
    "paths": {
      "@nx-demo-workspace/shared-models": ["libs/shared-models/src/index.ts"],
      "@nx-demo-workspace/frontend": ["apps/frontend/src/app/app.tsx"]
    }
  }
}

```

Open `apps/backend/src/main.ts` and add this illegal import at the top of the file:

```

// Illegal cross-boundary import
import { UserProfile } from '@nx-demo-workspace/frontend';

```

Step 6.5: Run the Linter to Catch the Violation

Execute the workspace lint check. Nx will scan the project graph, evaluate the code imports against your `depConstraints` tags, and catch the architectural error before it ever reaches production.

```
pnpm nx lint backend
```

Observe the terminal output. The linter will throw a hard error explicitly stating: A project tagged with "scope:server" can only depend on libs with tags ["scope:shared", "scope:server"]

Step 6.6: Fix the Violation

Remove the illegal frontend import line from `apps/backend/src/main.ts` and re-run the linter to confirm the workspace is clean and compliant again:

```
pnpm nx lint backend
```

The command will now pass with zero errors.

Lab 7: Target Customization & Overrides via Plugins

Nx version 19+ uses Project Crystal to automatically infer targets (like build, serve, preview, test, lint) directly from tools like Vite, ESLint, or Jest. In this lab, you will learn how to globally customize and override these inferred settings without adding boilerplate `project.json` files to your sub-projects.

Step 7.1: Inspect the Inferred Configuration

Before overriding anything, inspect what Nx has automatically configured for your React frontend app based on your `vite.config.ts`.

```
pnpm nx show project frontend
```

Look at the output. You will see an inferred build target pointing to the `@nx/vite/plugin`.

Step 7.2: Locate the Global Plugin Configuration

Open your root `nx.json` file. You will see a `plugins` array where Nx registers active tooling adapters. It will look similar to this:

```
{  
  "plugins": [  

```



```
{
  "plugin": "@nx/vite/plugin",
  "options": {
    "buildTargetName": "build",
    "serveTargetName": "serve",
    "previewTargetName": "preview"
  }
}
```

Step 7.3: Globally Override Inferred Build Configurations

We will now customize the `@nx/vite/plugin` behavior globally. Let's force all Vite builds in the workspace to output to a specific unified distribution structure and automatically use strict caching options.

Update the `@nx/vite/plugin` entry in your `nx.json` file to match this structure:

```
{
  "plugins": [
    {
      "plugin": "@nx/vite/plugin",
      "options": {
        "buildTargetName": "production-build",
        "serveTargetName": "serve",
        "previewTargetName": "preview"
      }
    }
  ]
}
```

By changing `"buildTargetName"` to `"production-build"`, you are globally renaming the command required to compile Vite projects across the entire workspace.

Step 7.4: Verify the Global Overrides

Run the project query command again to see if Nx updated its graph architecture based on your new rules:

```
pnpm nx show project frontend
```

Notice that the build target is gone, and a new target named `production-build` has taken its place.

Test your renamed build command:

```
pnpm nx production-build frontend
```

Step 7.5: Fine-Tune Specific Local Target Settings

If you want to keep global configurations but override settings for a single project (e.g., passing a specific flag only to the frontend), you can inject a targets block directly under the projects section in your root nx.json.

Open `nx.json` and append the target override block to your existing frontend definition:

```
{
  "projects": {
    "frontend": {
      "tags": ["scope:client"],
      "targets": {
        "production-build": {
          "options": {
            "sourcemap": true
          }
        }
      }
    }
  }
}
```

Step 7.6: Audit the Final Merged Configuration Pipeline

To verify that your local override was properly merged into the plugin's global configuration, query the exact configuration layout using the terminal graph view:

```
pnpm nx show project frontend --web
```

This launches a local web view showing exactly how Nx merged your local sourcemap: true option with the global inferred configurations.

Lab 8 : Replace react.js app with next.js app

Step 1: Safely Remove the React App

Instead of deleting folders manually (which leaves broken references in `nx.json` or `tsconfig.base.json`), use the `@nx/workspace:rm` generator.

```
pnpm nx g @nx/workspace:rm frontend
```

- What this does: It deletes the `apps/frontend` folder and automatically scrubs any references to it out of your workspace configurations, including the tags and overrides you added in Lab 6 and Lab 7.

Step 2: Install the Nx Next.js Plugin

Add the official Next.js plugin to your workspace root:

```
pnpm add -D -w @nx/next
```

Step 3: Generate the Next.js Application

Scaffold the new frontend application using the Next.js generator. Run the following command:

```
pnpm nx g @nx/next:app apps/frontend --appDir=true --srcDir=false --e2eTestRunner=none
```

- `apps/frontend`: Keeps your folder structure consistent so you do not have to change directory names elsewhere.
- `--appDir=true`: Configures the modern Next.js App Router setup.

Step 4: Restore Workspace Tags & Configuration

Because the old frontend configuration was completely removed, you need to re-apply the architectural metadata tags used in Lab 6.

Open your root `nx.json` file, find your newly created `apps/frontend` entry under `projects`, and re-apply the `scope:client` tag:

```
{
  "projects": {
    "apps/frontend": {
      "tags": ["scope:client"]
    },
    "backend": {
      "tags": ["scope:server"]
    },
  },
}
```

```
"shared-models": {  
  "tags": ["scope:shared"]  
}  
}  
}
```

Step 5: Verify the Target Inference

Next.js targets (like build, dev, start) are automatically inferred from your project files by Project Crystal. Verify that Nx has mapped out the new architecture:

```
pnpm nx show project apps/frontend
```

You can now run your new Next.js application locally:

```
pnpm nx dev apps/frontend
```

Lab 9 : Configure the Next.js app to use the 'shared library'

To connect your new Next.js application to the shared TypeScript library (shared-models) created in Lab 4, you must configure Next.js to compile internal workspace packages. Modern versions of Next.js handle this seamlessly via the `transpilePackages` feature.

Step 1: Expose the Library inside Next.js Configuration

Because internal monorepo libraries exist as raw TypeScript source files outside of the Next.js application root, Next.js needs permission to compile them on the fly.

1. Open `apps/frontend/next.config.js`.
2. Update the configuration to include the `transpilePackages` array, referencing the exact npm alias of your shared library (found in your root `tsconfig.base.json` paths):

```
//@ts-check  
const { composePlugins, withNx } = require('@nx/next');  
/**  
 * @type {import('@nx/next/plugins/with-nx').WithNxOptions}  
 */const nextConfig = {  
  nx: {  
    svgr: false,  
  },  
  transpilePackages: ['@scope:shared'],  
};
```

```
// Add this line to tell Next.js to compile your workspace library
transpilePackages: ['@nx-demo-workspace/shared-models'],
};
const plugins = [
  withNx,
];
module.exports = composePlugins(...plugins)(nextConfig);
```

Step 2: Use the Shared Library in a Next.js Component

Now you can import your shared TypeScript interface directly into your Next.js pages or components.

1. Open `apps/frontend/src/app/page.tsx`.
2. Replace the boilerplate code with an implementation that consumes the `UserProfile` interface:

```
import { UserProfile } from '@nx-demo-workspace/shared-models';
export default function Index() {
  // Enforce types safely using the library from Lab 4
  const mockUser: UserProfile = {
    id: 'next-101',
    name: 'NextJS Administrator',
    role: 'admin',
  };

  return (
    <div style={{ padding: '2rem', fontFamily: 'sans-serif' }}>
      <h1>Welcome to the Next.js Frontend App</h1>
      <div style={{ border: '1px solid #ccc', padding: '1rem',
borderRadius: '8px' }}>
        <h3>Active User Profile (Typed):</h3>
        <p><strong>ID:</strong> {mockUser.id}</p>
        <p><strong>Name:</strong> {mockUser.name}</p>
        <p><strong>Role:</strong> {mockUser.role}</p>
      </div>
    </div>
  );
}
```

Step 3: Verify the Integrated Project Graph

Regenerate your visual project graph. Nx will scan the Next.js import statements, resolve the TypeScript path alias, and map out the new dependency instantly.

```
pnpm nx graph
```

In your browser window, you will now see a clean dependency line flowing from your new frontend Next.js node directly into the shared-models library node.

Step 4: Launch the Dev Server

Test the compilation pipeline by spinning up the Next.js development server:

```
pnpm nx dev frontend
```

Open <http://localhost:4200> in your browser to see your typed profile component render successfully.
